Kristianstad University
SE-291 88 Kristianstad
+46 44-250 30 00
www.hkr.se

Independent project (degree project), 15 credits, for the degree of
e.g. Bachelor of Arts in Education
Semester Year e.g. Spring Semester 2026
Faculty of Natural Science

Performance Impact of Authenticated Encryption Algorithms on Latency in Near Real-Time Message-Based Communication

Ian Onamu, Hugo Nilsson

Author

Ian Onamu, Hugo Nilsson

Title

Performance Impact of Authenticated Encryption Algorithms on Latency in Near Real-Time Message-Based Communication

Supervisor

Fredrik Jönsson

Examiner

Zeinab Shahbazi

Abstract

This thesis investigates how two authenticated encryption algorithms, AES-GCM and ChaCha20-Poly1305, affect end-to-end latency in near real-time message-based communication. The study uses a quantitative experimental design in which synthetic JSON messages of 128, 512, and 2048 bytes are transmitted in a controlled TCP-based client-server environment. Each message is encrypted, transmitted, verified, decrypted, and acknowledged at the application level. In total, 60,000 latency measurements were collected and analyzed using descriptive statistics, confidence intervals, and Welch's t-test.

The results show that AES-GCM achieved lower mean end-to-end latency than ChaCha20-Poly1305 across all tested message sizes. AES-GCM achieved approximately 15-20 percent lower mean latency in the tested environment, which is likely related to AES-NI hardware acceleration on the AMD Ryzen 7 9700X processor. At the same time, both algorithms demonstrated very low sub-millisecond response times, indicating that both are practically suitable for near real-time communication. The study indicates that AES-GCM is particularly suitable in modern x86 environments with AES-NI support, while ChaCha20-Poly1305 continues to be relevant in environments without hardware acceleration or where software-based performance and energy efficiency are important.

Keywords

AES-GCM, ChaCha20-Poly1305, AEAD, latency, near real-time communication, encryption performance

Table of Content

List of Abbreviations

Abbreviation	Full Term
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
AES-GCM	Advanced Encryption Standard – Galois/Counter Mode
AES-NI	Advanced Encryption Standard New Instructions
API	Application Programming Interface
ARM	Advanced RISC Machine
CCM	Counter with CBC-MAC
CTR	Counter Mode
GCM	Galois/Counter Mode
GHASH	Galois Hash Function
IETF	Internet Engineering Task Force
IoT	Internet of Things
JSON	JavaScript Object Notation
MAC	Message Authentication Code
NIST	National Institute of Standards and Technology
RFC	Request for Comments
TCP	Transmission Control Protocol
TLS	Transport Layer Security
WAN	Wide Area Network
x86	32/64-bit processor architecture (Intel/AMD)

1. Introduction

1.1. Background

Modern networked systems require continuous, near real-time communication. Guaranteeing data security, specifically confidentiality integrity and availability is essential but introduces a fundamental engineering challenge: balancing robust cryptographic protection with strict latency constraints. Cryptographic operations consume CPU cycles and memory, which can lead to processing bottlenecks and increased end-to-end latency [1]. To secure data over public networks, systems typically rely on protocols like Transport Layer Security (TLS). The evolution to TLS 1.3 streamlined the cryptographic handshake to reduce connection delays and strictly limited cipher suites to Authenticated Encryption with Associated Data (AEAD) algorithms [1]. Understanding the performance characteristics of these algorithms is therefore vital for minimizing system delays.

AEAD is a symmetric cryptographic paradigm that provides confidentiality for a secret message while simultaneously ensuring the authenticity of both the message and its unencrypted metadata (associated data) [2] [3]. A detailed explanation of AEAD is provided in Section 2.1. Associated data often includes routing headers that must remain in plaintext to direct network traffic but require strict protection against tampering [2].

AEAD is used in modern secure communication because traditional unauthenticated encryption can be malleable: an attacker who modifies ciphertext in transit may cause predictable changes in the decrypted plaintext, enabling message manipulation [2]. AEAD prevents this by generating a mathematical authentication tag (a digital seal) alongside the ciphertext [3]. During decryption, if the mathematical tag verification fails, the packet is completely rejected. However this dual requirement of encrypting data and generating an authentication tag imposes a notable computational burden on the system [3].

The latency of an AEAD scheme is heavily dictated by its architectural design. This study focuses on two prominent algorithms: AES-GCM and ChaCha20-Poly1305, of which we will motivate further in section 1.2.

1.2. AES-GCM vs ChaCha20-Poly1305

AES is a block cipher that processes data in fixed-size blocks (128 bits) [2]. AES-GCM (Advanced Encryption Standard in Galois/Counter Mode) is a mode of operation that combines the AES block cipher with an authentication mechanism to provide Authenticated Encryption with Associated Data (AEAD) security properties. In this construction, AES is used in counter (CTR) mode for encryption, while authentication is achieved through the Galois hash function (GHASH) [2]. It is highly parallelizable and can benefit from dedicated hardware acceleration, such as the AES-NI instruction set found in modern processors, to achieve extremely fast execution times [1], [3]. However, it is important to distinguish between general claims and implementation context: AES can also be efficient in software, but the relative advantage of AES-GCM may vary depending on hardware capabilities, compiler optimizations, and library implementations [4].

To address these limitations, ChaCha20-Poly1305 was introduced as an alternative AEAD construction optimized for software implementations [3]. Similar to AES-GCM, the scheme combines an encryption algorithm with an authentication mechanism. ChaCha20 is used as the stream cipher responsible for encryption, while Poly1305 provides message authentication and integrity verification. According to its designers, the algorithm relies on simple arithmetic and logical operations, allowing efficient execution without relying on specialized hardware instructions [3], [4].

In networked communication systems overall latency is influenced by several factors, including message size, network stack processing, buffering and transmission conditions. Cryptographic operations represent one component of this latency and introduce additional computational overhead during message processing. Evaluating these algorithms therefore requires decomposing their overhead into measurable system metrics. Computational latency represents the CPU time required to encrypt a message and generate its authentication tag, which directly affects the responsiveness of the system [3], [4].

Differences in execution speed stem primarily from the architectural choice between a hardware-accelerated block cipher and a software-optimized stream cipher. Additionally transmission overhead is introduced by the encryption process itself; appending the authentication tag expands the overall size of every transmitted message, which incrementally increases the required network bandwidth [4]. While previous research

often evaluates cryptographic performance during large data transfers, there is a critical need to understand the latency impact when processing small, frequent messages typical of near real-time communication.

To provide a structured overview of the two algorithms under study, Table 1 summarizes their key characteristics across the dimensions most relevant to this thesis: design philosophy, hardware dependency, performance context and deployment environment.

Table 1: *Comparison of AES-GCM and ChaCha20-Poly1305*

Characteristic	**AES-GCM**	**ChaCha20-Poly1305**
Algorithm Type	Block cipher (AES) in Galois/Counter Mode [2]	Stream cipher (ChaCha20) with Poly1305 MAC [5]
AEAD Construction	AES-CTR for encryption + GHASH for authentication[2]	ChaCha20 for encryption + Poly1305 for authentication [5]
Key Size	128-bit, 192 bit or 256-bit	256-bit
Authentication Tag Size	128-bit [2]	128-bit [5]
Hardware Dependency	High, relies on AES-NI instruction set for optimal performance [3]	Low, designed for efficient software-only execution [3]
Performance on AES-NI Hardware	Very fast (~1–2 cycles/byte) [3]	Moderate (~8 cycles/byte) [3]
Performance without AES-NI	Significantly slower[3]	Consistently fast regardless of hardware[3]
Parallelizability	High, supports parallel encryption blocks [2]	Limited, sequential stream cipher design [5]
Primary Use Case	High-performance servers, TLS 1.3, cloud infrastructure[6]	Mobile devices, IoT, embedded systems, software-only environments [4]
Standardization	NIST SP 800-38D [2]	RFC 8439 (IETF) [7]
Designed By	Joan Daemen and Vincent Rijmen [2]	Daniel J. Bernstein [8]

Characteristic	AES-GCM	ChaCha20-Poly1305
Vulnerability to Nonce Reuse	Critical, nonce reuse breaks security completely[2]	More resilient, better resistance to nonce misuse [7]

As shown in Table 1, the two algorithms represent fundamentally different design philosophies. AES-GCM achieves its performance advantage through hardware acceleration, making it the optimal choice for modern server and desktop environments equipped with AES-NI support. ChaCha20-Poly1305 by contrast was specifically designed by Bernstein [8] to deliver consistent performance in pure software implementations, making it particularly well-suited for resource-constrained or heterogeneous environments where hardware acceleration cannot be guaranteed. Both algorithms are mandated cipher suites in TLS 1.3 [6], which underscores their relevance to modern secure communication systems and motivates their selection as the subject of this comparative study.

1.3. Problem Statement

As the demand for secure real-time communication continues to grow, developers face the challenge of maintaining both speed and data protection. Authenticated encryption ensures confidentiality and integrity but may slow down data transmission and increase the processing load on devices. The performance impact of different encryption algorithms, especially in real-time applications is not always clearly understood. AES-GCM and ChaCha20-Poly1305 are commonly used in modern communication systems. However, empirical evidence regarding their sustained end-to-end latency performance in continuous near real-time message-based communication remains limited. Existing research often focuses either on raw computational benchmarks or protocol-level handshake performance, leaving uncertainty about how these algorithms perform during ongoing message exchange in established secure connections. This raises the question of whether a measurable and statistically significant difference exists in end-to-end latency between AES-GCM and ChaCha20-Poly1305 in such environments.

1.4. Research Question(s)

How does end-to-end latency differ between AES-GCM and ChaCha20-Poly1305 in near real-time message-based communication?

Hypotheses

This study is guided by the following hypotheses:

H_0 (Null hypothesis): There is no statistically significant difference in mean end-to-end latency between AES-GCM and ChaCha20-Poly1305 in near real-time message-based communication.

H_1 (Alternative hypothesis): There is a statistically significant difference in mean end-to-end latency between AES-GCM and ChaCha20-Poly1305 in near real-time message-based communication.

1.5. Scope and Limitations

This study focuses on the relationship between data encryption and latency in real-time network communication. The scope is limited to analysing commonly used encryption algorithms under controlled network conditions. Factors such as packet loss, network congestion and hardware differences are acknowledged but not deeply examined, as they fall beyond the focus of this study.

1.6. Significance of the Study

Ensuring both speed and security in real-time network communication systems is a critical challenge in modern computer science. The findings of this study will provide valuable insights into how encryption algorithms affect the performance of such systems. By comparing AES-GCM and ChaCha20-Poly1305, this study contributes to a clearer understanding of potential performance differences between widely adopted authenticated encryption algorithms in latency-sensitive environments.

The results can help software engineers and network developers make more informed design choices when selecting authenticated encryption algorithms for latency-sensitive applications, such as video conferencing, online gaming, instant messaging and live data streaming. Additionally, the study can serve as reference for future research exploring optimization techniques for secure real-time communication.

1.7. Thesis Structure

The remainder of this thesis is structured as follows. Chapter 2 presents the literature review, including the theoretical background, previous research, and the identified research gap. Chapter 3 describes the methodology and experimental setup used to evaluate the latency of the selected encryption algorithms. Chapter 4 presents the experimental results together with the statistical analysis of the collected data. Chapter 5 discusses the findings and compares them with previous research. Finally, Chapter 6 summarizes the conclusions of the study and outlines possible directions for future work.

1.8. Declaration

We hereby confirm that this thesis is our own original work. AI tools were used solely for language refinement.

2. Technical Background

2.1. AEAD

An AEAD-algorithm is built up of two main operations: authenticated encryption and authenticated decryption. During encryption, the algorithm takes a secret key as input together with a nonce, plaintext and optional associated data. The associated data is transmitted in plaintext but is included in the authentication process to ensure its integrity. This is typically used for information that must remain visible for routing or protocol management such as IP addresses, TCP headers, or TLS record sequence numbers, but still needs protection against tampering [9]. The encryption process produces ciphertext together with an authentication tag. During decryption the authentication tag is verified before the plain text is recovered. This unified structure ensures both confidentiality and integrity within a single cryptographic construction [10].

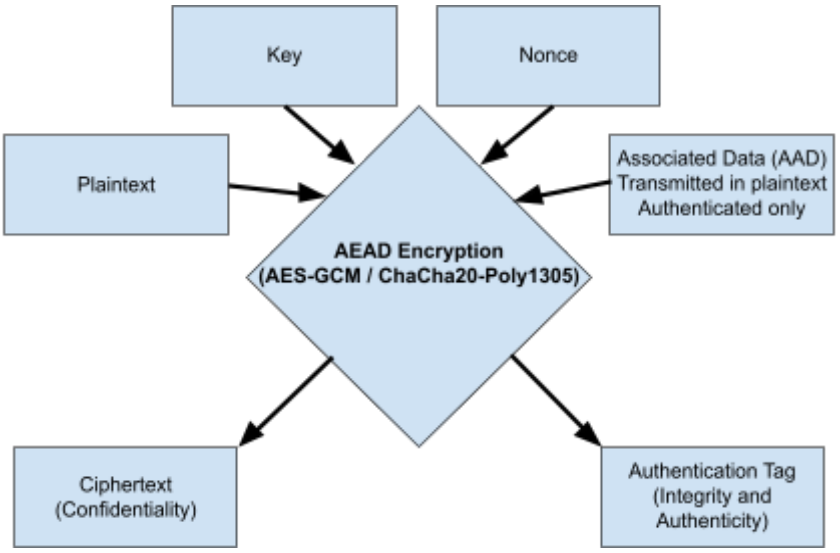

Figure 1. *General structure of an Authenticated Encryption with Associated Data (AEAD) process. Plaintext is encrypted to produce ciphertext, while associated data (AAD) is transmitted in plaintext but included in the authentication computation. The algorithm outputs both ciphertext and an authentication tag ensuring confidentiality, integrity, and authenticity.*

As illustrated in Figure 1, the AEAD encryption function takes plaintext, a nonce, optional associated data (AAD), and a secret key as input. The plaintext is encrypted to produce ciphertext, while the associated data is not encrypted but is included in the authentication computation. The algorithm outputs both ciphertext and an authentication tag, ensuring the integrity and authenticity of the encrypted message.

2.2. GHASH vs Poly1305

Although AES-GCM and ChaCha20-Poly1305 both follow the general AEAD structure, they use different mechanisms to compute the authentication tag. In AES-GCM, authentication is provided by GHASH, a universal hash function based on multiplication in a binary Galois field. GHASH is used in GCM to authenticate both the ciphertext and the additional authenticated data, while the plaintext is protected for confidentiality through counter-mode encryption [2].

ChaCha20-Poly1305 instead uses Poly1305 as its message authentication mechanism. RFC 8439 defines ChaCha20 as the stream cipher and Poly1305 as the authenticator used together in the combined AEAD construction [7]. Poly1305 computes an authentication tag through polynomial evaluation and is designed to be efficient in software implementations [11].

This difference is relevant to the present study because authentication tag generation is part of the total cryptographic processing performed for each message. Since the experiment measures end-to-end latency, the measured performance includes both encryption and authentication operations.

2.3. Client Server Communication

The client-server communication model describes how an application message moves from a client to a server over a TCP connection. Before transmission, the application processes the message and applies an authenticated encryption scheme. The encrypted message is then sent over the TCP connection, which provides reliable delivery at the transport layer[6]. Once the server receives the message, the application verifies the authentication tag and decrypts the ciphertext before processing the plaintext.

In AEAD-based communication, encryption and authentication tag generation is done before transmission, while verification and decryption is done when the message is received. Since these operations are a part of the processing, the selected AEAD algorithm directly influences the end-to-end latency. This model helps explain where cryptographic operations introduce delay in near real-time message flows. The communication flow used in the experiment is illustrated in Figure 2.

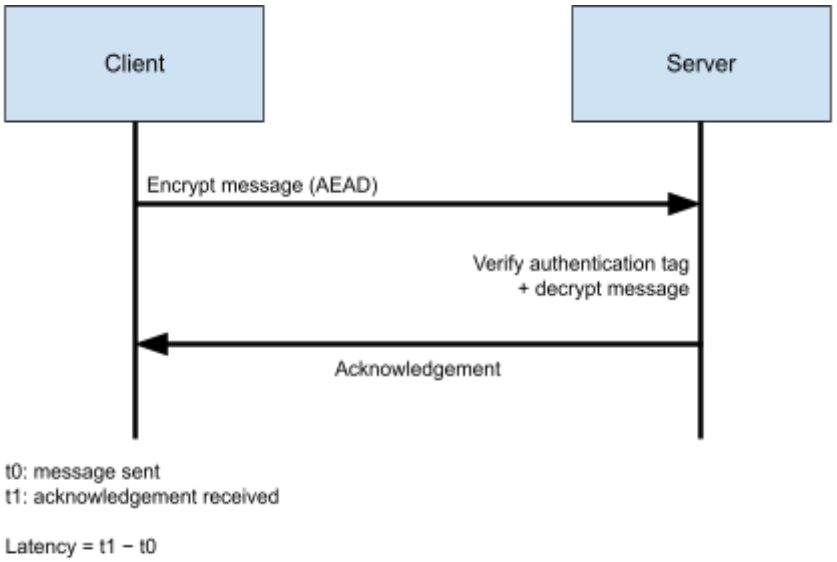

Figure 2. *Client–server communication flow used in the experiment. The client encrypts a message using AEAD (AES-GCM or ChaCha20-Poly1305) and sends the encrypted message to the server. The server verifies the authentication tag, decrypts the message, and returns an application-level acknowledgement. End-to-end latency is measured as the time between sending the encrypted message and receiving the application-level acknowledgement.*

2.4. Network Latency

Network latency is often explained as the total time from sender to receiver, which can be divided into processing delay, transmission delay, propagation delay, and queuing delay. In a controlled local network environment, transmission and propagation delays remain relatively stable.

Therefore differences in measured end-to-end latency can primarily be attributed to variations in processing delay introduced by the selected encryption algorithm. This model provides the theoretical basis for isolating the latency impact of AES-GCM and ChaCha20-Poly1305.

3. Literature Review

3.1. Systematic Literature Search

To support the literature review, a systematic literature search was conducted to identify relevant sources related to authenticated encryption, latency and near real-time message-based communication. The purpose of the search was to locate both theoretical and empirical studies that could help explain the performance characteristics of AES-GCM and ChaCha20-Poly1305 and to identify possible research gaps in the existing literature.

In the initial stage of the literature search, AI-assisted academic search tools were used to support an exploratory investigation of the research domain. In particular, tools designed for searching and summarizing scientific literature (e.g.such as Consensus) were used to identify relevant publications, extract key concepts, and refine search queries related to authenticated encryption and latency in communication systems. The use of AI-assisted search enabled a broader mapping of the research area by highlighting commonly used terminology, recurring research themes, and relationships between topics. This process helped identify important keywords such as AES-GCM, ChaCha20-Poly1305, AEAD, latency, performance, real-time communication, and end-to-end latency.

It is important to note that the AI-assisted search was used primarily as a support mechanism for discovery and keyword generation. All identified sources were subsequently verified through manual review, where their relevance, credibility, and methodological quality were assessed before inclusion in the study.

Through this initial exploration, central terms such as authenticated encryption, AEAD, AES-GCM, ChaCha20-Poly1305, latency, performance, real-time communication, TLS 1.3, end-to-end latency, and message-based systems were identified as relevant to the study.

In the second stage, a structured manual search was conducted using academic search tools and university-accessible databases. Searches were performed through Kristianstad University's library resources and relevant scientific databases such as Google Scholar, IEEE Xplore, and other accessible indexing platforms. Different combinations of keywords were tested and gradually refined in order to balance breadth and relevance. Example search strings included combinations such as:

("AES-GCM" OR "ChaCha20-Poly1305") AND latency

("authenticated encryption" OR AEAD) AND performance

("AES-GCM" AND "ChaCha20-Poly1305") AND ("real-time" OR "message-based communication")

("TLS 1.3" AND latency AND cryptography)

The search was initially broad in order to identify a wide range of potentially relevant material. The initial database searches across Google Scholar and IEEE Xplore yielded approximately 145 results. These results were then narrowed down by reviewing titles, abstracts, keywords, publication type and methodological relevance. Priority was given to peer-reviewed journal articles, conference papers, official cryptographic standards and technically authoritative sources directly related to authenticated encryption and performance evaluation.

To ensure relevance to the aim of this study, inclusion and exclusion criteria were applied. Sources were included if they addressed one or more of the following: authenticated encryption algorithms, AES-GCM, ChaCha20-Poly1305 latency or performance evaluation, secure communication protocols, or real-time / near real-time communication contexts. Sources were excluded if they focused on unrelated encryption algorithms, lacked sufficient methodological transparency or addressed security topics without relevance to latency, communication performance, or practical implementation (see Figure 3). After applying these criteria and removing duplicates, 18 full-text articles were reviewed of which 5 core empirical studies were ultimately retained for the final literature review.

Not all identified sources were included in the final review. Although several publications were scientifically credible, some were excluded because they focused on architectures, protocols, or evaluation settings that were too different from the TCP-based client-server setup used in this thesis. The final set of sources was therefore selected based not only on scientific quality, but also on direct relevance to the research question and the methodological scope of the study.

In addition to peer-reviewed academic literature, selected official technical standards and authoritative documents were included where appropriate. These sources were used primarily to define the cryptographic constructions under study, such as AES-GCM and

ChaCha20-Poly1305, and to provide foundational technical context where peer-reviewed sources alone were insufficient.

To capture practical performance metrics and identify relevant grey literature, the search strategy was expanded beyond traditional academic databases to include general web search engines. Targeted queries such as 'TLS 1.3 handshake performance benchmarks' and 'real-world latency AES-GCM ChaCha20', were utilized to discover technical industry reports and independent benchmark analyses. These sources, including industry evaluations of large-scale network deployments, were carefully selected to bridge the gap between academic theory and practical infrastructure constraints. By integrating this grey literature, the study achieves a more comprehensive view of how these encryption algorithms perform under real-world operational conditions, which directly complements the theoretical frameworks established by the peer-reviewed studies.

The systematic search and selection process is illustrated in Figure 3.

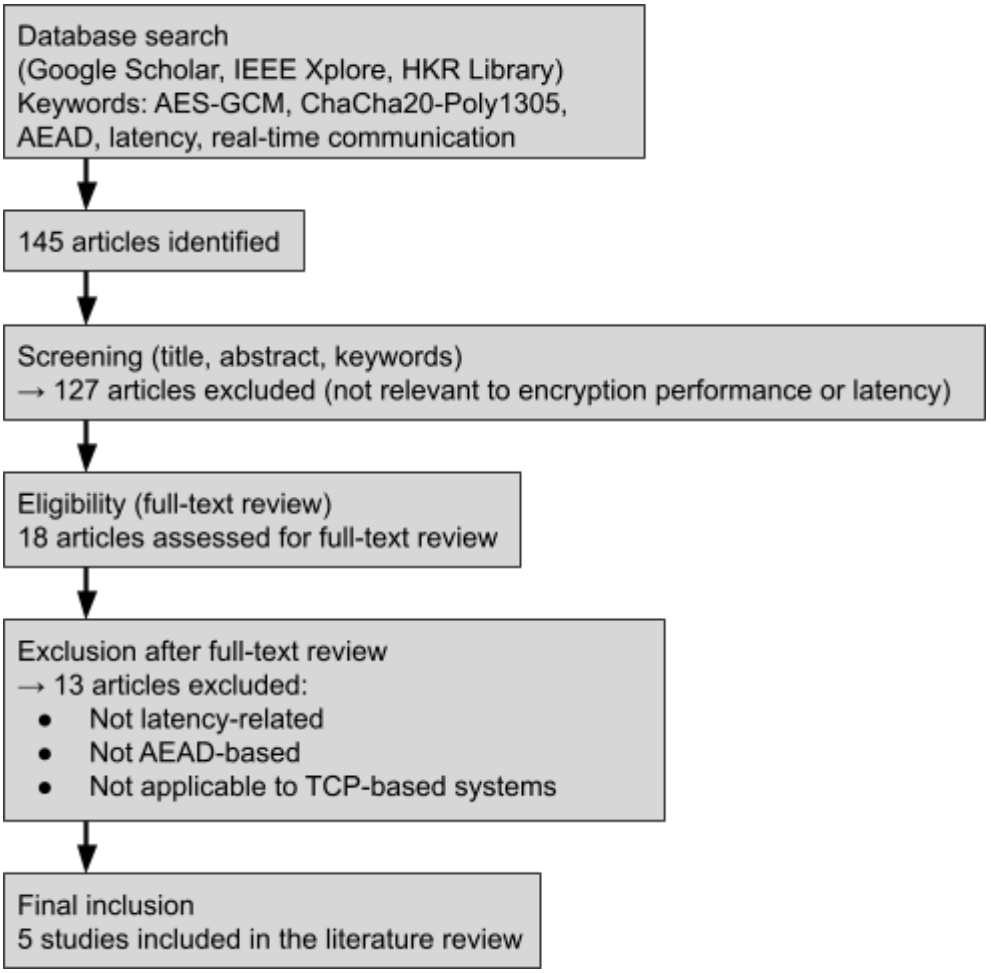

Figure 3. *Systematic literature search and selection process.*

3.2. Review of Previous Works

The performance and latency of authenticated encryption algorithms have been extensively studied across various domains, ranging from raw algorithmic benchmarking to protocol-level network simulations. This chapter reviews prior empirical studies on the performance of Authenticated Encryption with Associated Data (AEAD) schemes, specifically focusing on AES-GCM and ChaCha20-Poly1305. It analyzes the methodologies employed in these studies to highlight the existing gaps regarding continuous end-to-end latency in near real-time, message-based communication. Table 2 summarizes the most relevant literature, highlighting the methodologies used, the algorithms evaluated, the key findings and the limitations of each study. Sources that describe the fundamental design and standardization of the algorithms (such as NIST SP 800-38D [2] and RFC 8439) [7] are discussed in the theoretical background (Section 2.1 and 2.2) and are not repeated here.

Table 2. *Summary of selected studies on the performance of AES-GCM and ChaCha20-Poly1305 in network communication.*

Source	**Year**	**Algorithms Evaluated**	**Methodology & Context**	**Key Findings**	**Limitations**
Himmelberger [3]	2024	AES-GCM, ChaCha20-Poly1305, Deoxys II, and 5 others	Benchmarked raw CPU cycles per byte for 8 AEAD schemes on Intel Core i7 with AES-NI.	AES-GCM: ~1.6 cycles/byte; ChaCha20-Poly1305: ~8.2 cycles/byte.	Isolated CPU benchmark; no network transmission or real-time context.
Zeidler et al. [1]	2024	AES-GCM, AES-CCM, ChaCha20-Poly1305	Evaluated TLS 1.3 cipher suite overhead in a 5G Core control plane environment.	TLS adds <1% overhead in steady state; AES-GCM was the fastest cipher suite.	Focused exclusively on 5G infrastructure; not applicable to general TCP communication.
Alotaibi et al. [4]	2025	AES-128, ChaCha20, RC4, and lightweight ciphers	Tested energy efficiency and latency in IoT systems (Bluetooth mesh, ARM Cortex-M).	AES-128 was fastest (~0.05 ms) but consumed the most memory; ChaCha20 offered better energy efficiency.	Focused on low-resource IoT devices; not applicable to high-performance desktop environments.

Source	Year	Algorithms Evaluated	Methodology & Context	Key Findings	Limitations
Ragnarsson & Mallett [12]	2026	ChaCha20-Poly1305	Performance testing on Raspberry Pi and Mini PC for Industrial Control Systems (ICS/SCADA).	ChaCha20-Poly1305 is effective for real-time communication (57–60 µs latency on Mini PC).	Did not compare against AES-GCM; focused on embedded platforms without AES-NI.
AbdAllah et al. [13]	2020	AES (ECB, CBC, OFB, CFB, CTR modes)	Analyzed AES-NI hardware acceleration performance and power consumption on Intel processors.	AES-NI achieves significantly better performance and lower power consumption than software-only AES.	Did not evaluate AEAD modes (GCM) or compare with stream ciphers such as ChaCha20.

3.2.1. Handshake and Protocol-Level Latency

Other empirical investigations shift attention away from raw cryptographic execution and instead examine latency introduced by secure communication protocols. Meanwhile a study named *Performance Evaluation of Transport Layer Security in the 5G Core Control Plane* analyzed the performance overhead of Transport Layer Security (TLS) within the 5G Core control plane [1]. Their experimental setup employed Open5GS and UERANSIM to locally emulate a 5G network environment, measuring the time required for User Equipment (UE) to complete registration and session establishment procedures [1]. By isolating the contribution of TLS 1.2 and TLS 1.3 handshakes, the study found that TLS connection establishment increases total latency by approximately 28 % to 34 % during an initial (“cold start”) connection, while contributing less than 1 % overhead once the system reaches a steady operational state.

Another study *TLSv1.2 vs TLSv1.3: A Deep Dive into Handshake Efficiency | GO-EUC* conducted a large-scale network-level evaluation of TLS handshake efficiency by comparing TLS 1.2 and TLS 1.3 [14]. Using a custom benchmarking tool named *Achilles*, the study generated hundreds of thousands of unique client connections targeting a load balancer, enabling precise measurement of handshake Round-Trip Time (RTT) across multiple cipher suites, including AES-GCM and ChaCha20-Poly1305. The results

empirically demonstrated that TLS 1.3 handshakes are approximately 56% more efficient than those of TLS 1.2, primarily due to the reduced number of packet exchanges required.

While these studies successfully incorporate network behavior into their analyses, their methodologies predominantly focus on the connection establishment phase of secure communication [1],[14]. Although handshake latency is critical for initial connections, it does not reflect the continuous operational phase of near real-time systems. In many modern applications, connections persist over time, making the per-message encryption, authentication, and transmission latency of the symmetric cipher the dominant performance concern, an aspect not captured by handshake-centric evaluations.

3.2.2. Message-Oriented Security Architectures

Directly addressing real-time messaging scenarios, Lu et proposed [15] a decentralized, end-to-end hybrid encryption framework tailored for Instant Messaging Systems (IMS). Their evaluation compared Chinese national standard algorithms (SM2, SM3, and SM4) with conventional RSA and AES implementations. Network traffic was captured using Wireshark to verify that the system maintained data confidentiality and integrity while enabling client-to-client communication without reliance on a centralized authentication server 12.

3.3. Research Gap

A synthesis of the reviewed literature reveals a clear methodological gap. Existing empirical studies tend to adopt one of two primary approaches. Some studies focus on isolated measurements of cryptographic execution costs at the CPU level in fully local environments [4], [3]. While such experiments provide detailed insights into algorithmic efficiency, the cryptographic operations are typically executed entirely in memory without transmitting ciphertext over a physical or emulated network. Consequently, these studies do not account for network-related factors such as TCP stack processing, transmission overhead, or network jitter.

Other investigations instead evaluate latency at the protocol level, particularly during the connection establishment phase of secure communication [1], [14]. These studies analyze the overhead introduced by cryptographic handshakes such as those in TLS and provide valuable insights into connection setup performance. However, they do not capture the

sustained latency characteristics of continuous message exchange within an already established secure connection.

Some research has also explored secure messaging architectures in real-time communication systems [15]. Although such work operates closer to practical messaging environments, it often relies on customized hybrid architectures combining asymmetric and symmetric cryptography. As a result, the resulting performance measurements are difficult to generalize to standard TCP-based deployments of widely used AEAD algorithms such as AES-GCM and ChaCha20-Poly1305.

As a result, empirical evidence addressing the sustained end-to-end latency of transmitting small, frequent messages over an already established secure connection remains limited.

The methodology employed in this thesis is designed to address this gap. By implementing a live TCP-based client–server testbed in Python, the study moves beyond isolated cycle-counting and handshake-focused evaluations. Instead, it measures holistic end-to-end latency from the moment a plaintext message is encrypted at the client, transmitted across the network, and subsequently decrypted and authenticated at the server. This approach captures the combined effects of computational execution, ciphertext expansion, and network transmission, thereby reflecting the operational characteristics of near real-time, message-oriented communication systems more accurately.

4. Method

4.1. Research Design

This study adopts a quantitative experimental research design. The goal is to measure how two authenticated encryption algorithms AES-GCM and ChaCha20-Poly1305 affect end-to-end latency in a near real-time communication environment. The experiment isolates encryption algorithm choice as the only manipulated factor and evaluates its measurable impact on latency. This controlled experimental setup is suitable for observing how the system behaves under different cryptographic configurations.

The experiment follows a controlled setup in which identical messages are transmitted between a client and a server while switching only the authenticated encryption algorithm between AES-GCM and ChaCha20-Poly1305. All other conditions, network environment, message size, hardware, and software configurations, remain constant to ensure that differences in performance can be attributed solely to the algorithm. The test environment simulates a near real-time communication scenario where encryption-related processing delay may influence overall system responsiveness.

4.2. Experimental Setup

4.2.1. Hardware

The experiments were conducted on a desktop computer equipped with an AMD Ryzen 7 9700X processor (8 cores), 32 GB RAM, running Windows 11. All tests were executed on the same hardware platform to eliminate performance variability introduced by differences in system capabilities. Two Python programs were implemented: a client responsible for encrypting and transmitting messages, and a server responsible for receiving, authenticating and decrypting them. Both programs ran on the same machine and communicated through a local TCP socket connection, simulating a controlled client–server messaging environment while minimizing external network variability. By running both the client and server on the same machine, the experiment ensures that measured latency differences can be attributed primarily to the encryption algorithms rather than external network conditions.

4.2.2. Software

Python 3.13.12 was used as the primary programming language. The cryptography library was utilized to implement AES-GCM and ChaCha20-Poly1305 following the recommended security guidelines [16]. The client-server architecture was implemented using Python sockets over TCP. Specifically the AES-GCM and ChaCha20-Poly1305 implementations provided by the Python cryptography package. The source code used for the experiment is available in the project repository: github.com/HugNil/AES-GCM_vs_ChaCha20-Poly1305.

4.2.3. Network Environment

All experiments were conducted on a local network to minimize any external impact and uncontrolled network latency. No additional background traffic was introduced under the tests. The controlled environment allowed stable and replicable measurements. The same network conditions were maintained across all test runs to ensure comparability.

4.3. Variables

The independent variable in the study is the encryption algorithm (AES-GCM or ChaCha20-Poly1305).

The dependent variable is end-to-end latency, measured in nanoseconds and then converted to microseconds.

Control variables include message size, hardware configuration, network environment, and number of repetitions per test.

4.4. Data Collection Procedure

Data collection was conducted through a controlled experimental setup where AES-GCM and ChaCha20-Poly1305 were executed under identical conditions. The messages used in the tests were divided into three fixed-size categories to represent different near real-time scenarios: small (~128 bytes), medium (~512 bytes), and large (~2048 bytes). All of the messages followed the same structured format, JavaScript Object Notation (JSON), to ensure that only the message size differed between tests.

All tests were conducted on the same hardware and the same local network environment to eliminate variations caused by external factors. Before the actual measurements began,

a warm-up phase was performed to reduce effects related to initialization and cache build-up.

For each algorithm and message size, the same procedure and sequence was followed. In each iteration, the client initiated communication by encrypting and transmitting a message to the server over TCP. Upon receiving the message, the server verified the authentication tag, decrypted the payload, encrypted an acknowledgment message, and transmitted it back to the client.

Latency measurement began immediately before the client started encrypting the outgoing message and ended when the client received the encrypted acknowledgement from the server and successfully verified and decrypted it. The measured latency therefore represents the full round-trip application-level latency, including encryption, transmission, authentication verification and decryption on both the client and server sides. The difference between the start and stop timestamps therefore represents the complete end-to-end latency of the encrypted communication cycle, including encryption, transmission, authentication verification, and decryption on both sides. Python's high-resolution performance timer (time.perf_counter_ns()) was used to measure elapsed execution time in nanoseconds.

Each configuration was executed in five blocks of 2000 iterations, resulting in a total of 10,000 latency measurements per algorithm and message size to minimize the impact of random variations. To reduce systematic bias and temporal effects, algorithms were executed in alternating blocks. The measured latency for each iteration, along with its configuration parameters was logged and saved into individual CSV files before being merged into a single comprehensive dataset (all_results.csv) for analysis.

4.5. Data Generation & Description

The data used in this study is not sourced from an external dataset but is instead generated programmatically during the experiment itself. For each trial, the client generates a synthetic plaintext message of a fixed size either 128 bytes, 512 bytes, or 2048 bytes intended to simulate the types of small, frequent messages characteristic of near real-time communication systems. These message sizes were selected to reflect a realistic range of payloads encountered in message-oriented protocols, from short control signals to moderately sized data frames. Each message is encrypted using the designated algorithm (AES-GCM or ChaCha20-Poly1305), transmitted over a TCP connection to the server,

decrypted upon receipt, and the round-trip latency is recorded in microseconds. This process is repeated 10,000 times per algorithm per message size, yielding a total of 60,000 latency measurements across all experimental conditions. The generated data is stored in CSV format for subsequent statistical analysis.

4.6. Statistical Analysis

The gathered latency data were analysed using descriptive statistics, including the mean, standard deviation and confidence intervals. A block-level analysis was also conducted to ensure no systemic temporal anomalies occurred during the alternating execution blocks.

To determine whether the difference between AES-GCM and ChaCha20-Poly1305 was statistically significant, Welch´s independent sample t-test was applied to compare the mean latency between AES-GCM and ChaCha20-Poly1305. Welch´s t-test was selected because it is suitable for comparing independent group means when equal variances cannot be assumed [17]. To reduce bias caused by repeated measurements within the same experimental run, the Welch's t-test was applied to block-level mean latencies rather than individual latency measurements.

The null hypothesis (H_0) stated that there was no significant difference in mean latency between the two algorithms. The alternative hypothesis (H_1) stated that a significant difference in mean latency existed. The level of significance was set to $\alpha = 0.05$.

To account for potential temporal dependencies within the experiment, a block-level analysis was additionally performed. The 10,000 measurements per message size were aggregated into five blocks of 2,000 measurements each, and a secondary Welch's t-test was applied to the resulting block means. This approach confirms that the observed performance differences are consistent across the full duration of the experiment and are not attributable to warm-up effects or temporal drift.

5. Results

5.1. Latency Results

This section reports the measured end-to-end latency of AES-GCM and ChaCha20-Poly1305 across different message sizes. The results are based on repeated measurements in the controlled experimental setup described in Chapter 3. Table 3 presents the descriptive statistics of the measured end-to-end latency, including the mean latency and standard deviation for each algorithm and message size.

Table 3. *Descriptive statistics of the measured end-to-end latency for AES-GCM and ChaCha20-Poly1305 across different message sizes.*

Message Size (bytes)	AES-GCM Mean (µs)	AES-GCM Std Dev (µs)	ChaCha20-Poly1305 Mean (µs)	ChaCha20-Poly1305 Std Dev (µs)
128	32.15	13.89	38.05	20.63
512	31.97	13.63	39.94	21.99
2048	32.72	13.21	39.05	20.79

As shown in Table 3, AES-GCM consistently produced lower mean latency than ChaCha20-Poly1305 across all tested message sizes. For example at a message size of 128 bytes, AES-GCM achieves a mean latency of approximately 32.15 µs, compared to 38.05 µs for ChaCha20-Poly1305. A similar pattern is observed for message sizes of 512 bytes and 2048 bytes.

The difference between the algorithms remains relatively consistent across the evaluated message sizes, while the message size itself has only a limited effect on the observed latency values. Each reported value is based on 10,000 latency measurements per algorithm and message size. Confidence intervals were calculated using the standard error of the mean.

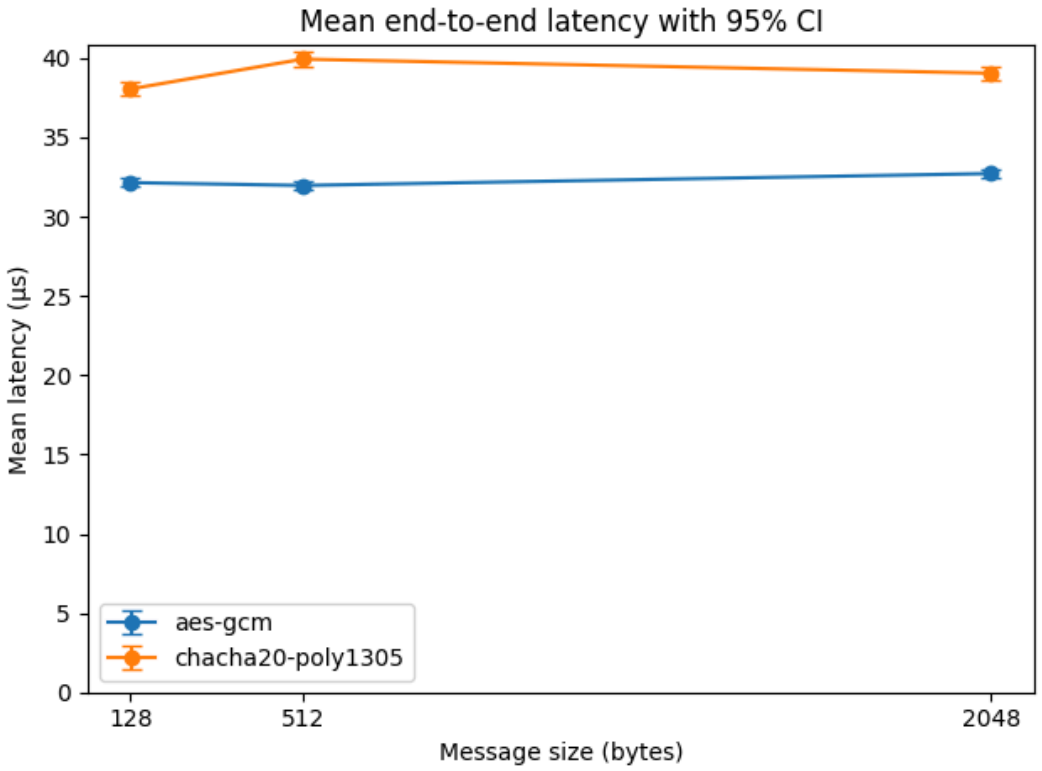

Figure 4. *Mean end-to-end latency for AES-GCM and ChaCha20-Poly1305 across different message sizes. Error bars represent 95% confidence intervals calculated from repeated measurements.*

Figure 4 illustrates the mean end-to-end latency for both encryption algorithms across the evaluated message sizes. The results demonstrate a clear and consistent separation between the two algorithms, where AES-GCM maintains lower latency across all tested configurations. The error bars represent the 95% confidence intervals and indicate relatively low variability in the measurements. Notably, AES-GCM exhibits narrower confidence intervals compared to ChaCha20-Poly1305, suggesting more stable and predictable performance.

While a slight variation in latency can be observed as message size increases, the overall impact of message size remains limited. The difference in mean latency between the algorithms remains consistent, ranging approximately between 6 and 8 microseconds across all tested message sizes.

5.2. Statistical Analysis

To determine whether the observed differences in latency between AES-GCM and ChaCha20-Poly1305 were statistically significant, a Welch's independent samples t-test was performed for each message size (128, 512, and 2048 bytes). The results of the statistical analysis are presented in Table 4.

The Welch's t-test was calculated using the following formula to account for unequal variances between the two algorithm groups:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

The degrees of freedom were approximated using the Satterthwaite equation. This rigorous mathematical approach ensures that the p-values reported in Table 4 are not influenced by differences in sample variance, providing a more conservative and reliable estimate of statistical significance.

Table 4. Welch's t-test results for latency differences across different message sizes.

Message size (bytes)	**t-value**	**p-value**	**Significant at α = 0.05**
128	-23.76	< 0.001	Yes
512	-30.79	< 0.001	Yes
2048	-25.68	< 0.001	Yes

For messages of 128 bytes, the test yielded $t = -23.76$ with $p < 0.001$. For 512 bytes, the result was $t = -30.79$ with $p < 0.001$, and for 2048 bytes, $t = -25.68$ with $p < 0.001$. In all cases, the p-values were substantially below the significance threshold, indicating strong evidence against the null hypothesis. Thus, the null hypothesis (H_0), which states that there is no difference in mean latency between AES-GCM and ChaCha20-Poly1305, is rejected in favor of the alternative hypothesis (H_1).

Additionally the statistical tests were performed independently for each message size, ensuring that the significance of the results is not influenced by aggregation across different configurations. The consistency of the statistical outcomes across all three message sizes reinforces the reliability of the observed performance difference.

A block-level analysis was also conducted to examine potential temporal variation within the experimental runs. The results showed the same overall trend as the main analysis, with AES-GCM generally achieving lower latency, but also indicated some variation between blocks. Therefore, the full-sample Welch's t-test remains the primary basis for the hypothesis test.

5.2.1. Distributional Analysis and Quantiles

To provide a more comprehensive view of the latency distributions beyond simple averages, Table 5 presents additional metrics including Median, 95th percentile (p95), 99th percentile (p99), and Maximum (Max) latency. These metrics are crucial for understanding the consistency and predictability of the algorithms in near real-time systems.

Table 5. Detailed Latency Statistics (Quantiles) for AES-GCM and ChaCha20-Poly1305.

Algorithm	Size (bytes)	Median	p95	p99	Max
AES-GCM	128	29.2 µs	47.0 µs	100.6 µs	208.6 µs
ChaCha20	128	30.6 µs	88.3 µs	124.8 µs	341.5 µs
AES-GCM	512	29.7 µs	34.9 µs	101.4 µs	247.4 µs
ChaCha20	512	31.3 µs	92.3 µs	117.8 µs	597.2 µs
AES-GCM	2048	30.2 µs	41.2 µs	100.0 µs	223.1 µs
ChaCha20	2048	31.7 µs	92.0 µs	126.8 µs	205.0 µs

The quantile data reveals that while the median values are relatively close, ChaCha20-Poly1305 exhibits higher p95 and maximum values compared to AES-GCM. For instance, at 512 bytes, the maximum latency for ChaCha20 reached 597.2 µs, which is more than double the maximum recorded for AES-GCM at the same size. This indicates that ChaCha20 is more prone to extreme latency spikes (outliers), whereas AES-GCM maintains a tighter and more predictable performance profile.

5.3. Comparison with State-of-the-Art Literature

To contextualize the experimental findings of this thesis, Table 6 presents a comparison between the latency results of the present study and performance metrics reported in recent state-of-the-art literature. While direct comparisons are challenging due to differences in hardware architecture, network environments, and measurement methodologies (e.g., CPU cycles per byte versus end-to-end microseconds), the relative performance trends between AES-GCM and ChaCha20-Poly1305 remain consistent across studies.

Table 6. *Comparison of latency results from the present study and selected previous studies.*

Study	Environment	Metric	AES-GCM	ChaCha20-Poly1305	Trend
The present study (2026)	AMD Ryzen 7 9700X (AES-NI), Local TCP Socket	End-to-end latency (µs)	~32.28 µs	~39.01 µs	AES-GCM ~15–20% lower mean latency, likely due to hardware acceleration
Himmelberger [3]	Intel Core i7 (AES-NI)	CPU cycles/byte	~1.6 cycles/byte	~8.2 cycles/byte	AES-GCM significantly faster at CPU level with AES-NI
Zeidler et al. [1]	5G Core (x86 servers)	TLS overhead	<1% overhead (fastest)	Slower than AES-GCM	AES-GCM optimal for high-performance server infrastructure
Alotaibi et al. [4]	IoT devices (ARM Cortex-M)	Latency (ms)	~0.05 ms (fastest)	Slower, better energy efficiency	AES-GCM faster, but ChaCha20 preferred for battery-powered devices
Ragnarsson & Mallett [12]	Mini PC (Industrial Control)	Functional time (µs)	Not evaluated	57–60 µs	ChaCha20-Poly1305 effective for real-time embedded communication

The comparison in Table 6 shows that the results of this experiment are consistent with findings from previous research. As demonstrated by Himmelberger and Zeidler et al., AES-GCM generally achieves lower latency than ChaCha20-Poly1305 on modern x86 architectures equipped with AES-NI hardware acceleration. Similarly, the present study measured lower end-to-end latency for AES-GCM than for ChaCha20-Poly1305 in the tested TCP-based client-server environment. Furthermore, the findings of Alotaibi et al. highlight that while AES-GCM provides lower latency performance in the tested environment, ChaCha20-Poly1305 remains highly relevant for environments where hardware acceleration is unavailable or energy efficiency is prioritized.

Furthermore it is important to address the discrepancy between our application-level results (15-20% lower mean latency) and raw CPU benchmarks, which often show AES-NI being 300-400% faster [3]. This is due to the 'dilution effect': in a real-world TCP socket, the pure encryption time is only one component of the total latency, which also includes network stack processing, OS overhead, and data transmission. Our study captures this practical reality, showing how hardware acceleration translates to actual end-to-end performance in a live messaging context.

6. Discussion

6.1. Interpretation

Overall, the statistical analysis provides strong evidence that AES-GCM offers lower latency performance within the tested environment. The experimental results demonstrate that AES-GCM achieved approximately 15% to 20% lower mean latency across all evaluated message sizes (128, 512, and 2048 bytes). This performance advantage is primarily attributed to the presence of AES-NI hardware acceleration on the AMD Ryzen 7 9700X processor used in the test environment, which reduces computational overhead compared to software-only execution [13].

It is important to note that in absolute terms, both AES-GCM and ChaCha20-Poly1305 demonstrated very low end-to-end latency throughout the experiment. The mean latency values ranged from approximately 32 µs to 40 µs across all tested message sizes and algorithms. These figures represent sub-millisecond response times, indicating that both algorithms are highly suitable for real-time communication from a practical standpoint. The observed 15–20% reduction in mean latency, while statistically significant, may therefore have limited practical impact in many application contexts where both algorithms would perform well within acceptable latency thresholds.

The introduction of quantile analysis further deepens this interpretation. By observing the gap between mean and median, and the substantially higher p99 and Max values for ChaCha20, we can conclude that AES-GCM is not only faster on average but also more consistent. In near real-time systems, the 'worst-case' latency is often a more critical metric than the average. Our data suggests that AES-GCM provides a more stable performance profile with fewer extreme outliers, making it highly suitable for applications where jitter and predictability are key concerns.

The data also reveals that message size had only a limited impact on the relative performance gap between the two algorithms. As the payload increases from 128 bytes to 2048 bytes, the absolute end-to-end latency remained relatively stable for both AES-GCM and ChaCha20-Poly1305. However, AES-GCM maintained a consistent performance advantage across all tested message sizes. This indicates that, in the tested environment, the choice of encryption algorithm had a greater effect on latency than the evaluated payload size. For real-time message-based TCP communication, where minimizing latency is critical to maintaining system responsiveness and user experience,

these findings suggest that AES-GCM is well suited when deployed on modern x86 architectures [1], [3], [13].

Conversely, while ChaCha20-Poly1305 exhibited higher latency and greater variation in this specific setup, it remains a relevant alternative in environments where AES-NI hardware acceleration is unavailable. The algorithm's software-only design ensures that its execution time is not dependent on specialized hardware instructions, which is a valuable characteristic for systems lacking dedicated cryptographic hardware [8]. In environments where AES-NI is unavailable, such as older processors or certain embedded systems, the software-only AES-GCM may perform less favorably than hardware-accelerated AES-GCM. In such scenarios, ChaCha20-Poly1305 provides a robust and efficient alternative, maintaining a high level of security without the severe performance degradation associated with software-based AES [7].

Furthermore, the consistency of the results across multiple iterations and blocks underscores the reliability of AES-GCM in continuous data streams. The statistical analysis, which yielded p-values of less than 0.001 for all message sizes, confirms that the observed latency differences are not the result of random system fluctuations or network jitter. While previous research has established the theoretical efficiency of AES-NI [13], this study provides empirical validation that these benefits are fully realized in application-level TCP sockets. The findings confirm that the choice of AEAD algorithm is a critical architectural decision that directly impacts the real-time capabilities of secure messaging systems. Developers must carefully weigh the target hardware environment against the latency requirements of their applications when selecting a cryptographic standard [10].

6.2. Comparison with Literature

The findings of the present study are generally consistent with previous research; however, the comparison must be interpreted with caution, as the selected studies use different performance metrics and experimental environments. As shown in Table 6, the present study measured end-to-end latency in a local TCP message-based setup, while previous studies reported metrics such as CPU cycles per byte, TLS overhead, latency in milliseconds, and functional execution time. Therefore the absolute values cannot be directly compared across studies. Unlike many previous studies that focus on isolated cryptographic benchmarks or protocol-level overhead, this study evaluates encryption

within an end-to-end message-based communication workflow, providing a more application-level perspective on latency.

Despite these methodological differences, the results point in a similar direction. Previous work has shown that AES-GCM performs strongly on x86-based systems with AES-NI support [1], [3]. This is consistent with the lower latency observed for AES-GCM in the present experiment and supports the interpretation that hardware acceleration plays an important role in AES-GCM´s performance advantage. This empirical validation confirms that the theoretical efficiency of AES-NI is fully realized in application-level TCP sockets, even when partially diluted by the overhead of the system stack and runtime environment.

At the same time, previous research also highlights that ChaCha20-Poly1305 remains relevant in environments where AES acceleration is unavailable or where software-only execution, energy efficiency, or embedded deployment are important [4], [12]. Therefore, the results of the present study should not be interpreted as showing that ChaCha20-Poly1305 is generally inferior, but rather that AES-GCM was better suited to the specific hardware and software environment used in this experiment.

6.3. Limitations

While this study provides empirical evidence of the latency differences between AES-GCM and ChaCha20-Poly1305, several limitations must be acknowledged. The experiments were conducted on a single hardware platform equipped with AES-NI hardware acceleration (AMD Ryzen 7 9700X), which means the results cannot be directly generalized to devices lacking such support such as embedded systems, older processors, or mobile hardware. Furthermore, the communication scenario was simulated using a local TCP socket (localhost), intentionally eliminating external network variables such as routing delays and packet loss. While this isolation was necessary to measure cryptographic latency accurately, it does not fully replicate real-world wide-area network conditions, where network-induced overhead may reduce the practical significance of the observed differences [18].

The study also focused exclusively on end-to-end latency as the primary performance metric. Other important factors such as CPU utilization, memory consumption and energy efficiency were not evaluated. For resource-constrained environments, such as IoT devices or battery-powered systems, these metrics may be equally or more important than

raw latency [4]. Future research should incorporate multi-platform testing and a broader set of performance indicators to provide a more complete evaluation of AEAD algorithm suitability across diverse deployment contexts.

7. Conclusion

7.1. Summary

This study investigated the end-to-end latency of two widely used Authenticated Encryption with Associated Data (AEAD) algorithms, AES-GCM and ChaCha20-Poly1305, in a continuous, message-based TCP communication scenario. The experiment was conducted on a local network using a client-server architecture implemented in Python, with synthetic JSON payloads of three sizes: 128, 512, and 2048 bytes. A total of 60,000 latency measurements were collected across five experimental blocks, and the results were analyzed using descriptive statistics and Welch's t-test.

The results consistently demonstrated that AES-GCM achieved lower end-to-end latency than ChaCha20-Poly1305 across all tested message sizes, with AES-GCM achieving approximately 15% to 20% lower mean latency. This advantage is attributed to the AES-NI hardware acceleration available on the test processor (AMD Ryzen 7 9700X). The statistical analysis confirmed that the observed differences are highly significant ($p < 0.001$), indicating that the performance gap is statistically reliable in the tested environment.

7.2. Contributions

This thesis makes several contributions to the existing body of knowledge on cryptographic performance evaluation. It provides empirical evidence of the latency impact of AES-GCM and ChaCha20-Poly1305 in a message-based TCP communication context, a scenario that has received limited attention in prior research, which has predominantly focused on raw throughput benchmarks or TLS handshake overhead. Secondly the study introduces a controlled and reproducible experimental methodology that isolates cryptographic latency from network-induced variables, offering a reusable framework for future performance evaluations. Third, the findings contribute practical guidance for developers and system architects selecting AEAD algorithms for real-time secure messaging applications on modern x86 hardware.

7.3. Social and Ethical Relevance

The social relevance of this work lies in its connection to secure and responsive digital communication systems. Applications such as messaging platforms, IoT systems, and real-time services depend on encryption methods that protect data without introducing unnecessary latency. By comparing AES-GCM and ChaCha20-Poly1305, this thesis provides practical insight for developers and system architects selecting encryption algorithms for systems where both data protection and low latency are important.

From an ethical perspective, the study does not involve human participants, personal data, or sensitive user information, since all experimental data was synthetically generated in a controlled local environment. However, responsible cryptographic algorithm selection remains important. Latency should not be the only selection criterion; security requirements, implementation context, hardware support, and established cryptographic recommendations must be considered.

7.4. Future Work

Several directions for future research emerge from the limitations of this study. First, the experiment should be replicated on diverse hardware platforms, including devices without AES-NI support, ARM-based processors and resource-constrained IoT hardware to provide a more comprehensive comparison of algorithm performance across different architectures. Secondly, future studies should extend the evaluation to real-world network environments, including wide-area networks and cloud-based deployments, to assess how network latency and packet loss affect the relative performance of the two algorithms. Third, additional performance metrics such as CPU utilization, memory consumption, and energy efficiency should be incorporated to support holistic algorithm selection in power-sensitive environments. Future work could also explore the performance of newer or alternative AEAD schemes, such as AES-GCM-SIV [19] or XChaCha20-Poly1305, which offer improved nonce-misuse resistance and may be better suited for specific deployment scenarios.

References

[1] O. Zeidler, J. Sturm, D. Fraunholz, and W. Kellerer, ‘Performance Evaluation of Transport Layer Security in the 5G Core Control Plane’, in *Proceedings of the 17th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, Seoul Republic of Korea: ACM, May 2024, pp. 78–88. doi: 10.1145/3643833.3656140.

[2] ‘NIST SP 800-38D, Recommendationfor Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC’. Accessed: Feb. 24, 2026. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-38d.pdf

[3] M. Himmelberger, ‘Performance Analysis of AEAD Schemes’.

[4] A. Alotaibi, H. Aldawghan, and M. Frikha, ‘Lightweight Cryptography for Energy-Conscious Authentication in IoT Systems.’, *Int. J. Adv. Comput. Sci. Appl.*, vol. 16, no. 11, p. 1081, Nov. 2025, doi: 10.14569/ijacsa.2025.01611103.

[5] V. Arya and K. Roy, ‘Performance Analysis of Encryption and Decryption Algorithm: A Systematic Review’, in *2025 1st International Conference on Radio Frequency Communication and Networks (RFCoN)*, Jun. 2025, pp. 1–6. doi: 10.1109/RFCoN62306.2025.11085299.

[6] E. Rescorla, ‘The Transport Layer Security (TLS) Protocol Version 1.3’, Internet Engineering Task Force, Request for Comments RFC 8446, Aug. 2018. doi: 10.17487/RFC8446.

[7] Y. Nir and A. Langley, ‘ChaCha20 and Poly1305 for IETF Protocols’, Internet Engineering Task Force, Request for Comments RFC 8439, Jun. 2018. doi: 10.17487/RFC8439.

[8] D. J. Bernstein, ‘ChaCha, a variant of Salsa20’.

[9] P. Rogaway, ‘Authenticated-encryption with associated-data’, in *Proceedings of the 9th ACM conference on Computer and communications security*, Washington, DC USA: ACM, Nov. 2002, pp. 98–107. doi: 10.1145/586110.586125.

[10] D. McGrew, ‘An Interface and Algorithms for Authenticated Encryption’, Internet Engineering Task Force, Request for Comments RFC 5116, Jan. 2008. doi: 10.17487/RFC5116.

[11] D. J. Bernstein, ‘The Poly1305-AES Message-Authentication Code’, in *Fast Software Encryption*, vol. 3557, H. Gilbert and H. Handschuh, Eds, in Lecture Notes in Computer Science, vol. 3557. , Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 32–49. doi: 10.1007/11502760_3.

[12] K. O. Ragnarsson and J. Mallett, ‘Performance Testing of ChaCha20-Poly1305 for Internet of Things and Industrial Control System devices’, Mar. 19, 2026, *arXiv*: arXiv:2603.19150. doi: 10.48550/arXiv.2603.19150.

[13] ‘Advanced Encryption Standard New Instructions (AES-NI) Analysis: Security, Performance, and Power Consumption | Proceedings of the 2020 12th International Conference on Computer and Automation Engineering’, ACM Other conferences. Accessed: Apr. 13, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3384613.3384648

[14] ‘TLSv1.2 vs TLSv1.3: A Deep Dive into Handshake Efficiency | GO-EUC’. Accessed: Feb. 24, 2026. [Online]. Available: https://www.go-euc.com/tls12-vs-tlsv13-a-deep-dive-into-handshake-efficiency/

[15] ‘Enhancing security in instant messaging systems with a hybrid SM2, SM3, and SM4 encryption framework - PMC’. Accessed: Feb. 24, 2026. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC12435676/

[16] ‘Authenticated encryption — Cryptography 47.0.0.dev1 documentation’. Accessed: Feb. 24, 2026. [Online]. Available: https://cryptography.io/en/latest/hazmat/primitives/aead/

[17] M. Delacre, D. Lakens, and C. Leys, ‘Why Psychologists Should by Default Use Welch’s t-test Instead of Student’s t-test’, *Int. Rev. Soc. Psychol.*, vol. 30, no. 1, Apr. 2017, doi: 10.5334/irsp.82.
[18] O. Abolade, A. Okandeji, A. Oke, M. Osifeko, and A. Oyedeji, ‘Overhead effects of data encryption on TCP throughput across IPSEC secured network’, *Sci. Afr.*, vol. 13, p. e00855, Sep. 2021, doi: 10.1016/j.sciaf.2021.e00855.
[19] S. Gueron and Y. Lindell, ‘GCM-SIV: Full Nonce Misuse-Resistant Authenticated Encryption at Under One Cycle per Byte’, 2015, 2015/102. Accessed: Feb. 14, 2026. [Online]. Available: https://eprint.iacr.org/2015/102